React + TypeScript · 10-Day Training Plan

Day 9 — Production Development

Your simple, friendly guide for today

COMING FROM	KnockoutJS / .NET (C#) → moving to React + TypeScript
TODAY'S GOAL	Work like a professional team: clean folder structure, a shared API layer, Git workflow & code reviews, coding standards, environment config, and getting the app ready to ship.

How to use this: Read **Section 1 (Learning Plan)** for each idea with a small example. Then read **Section 2 (Detailed Notes)** slowly, top to bottom — one read should make it click. Keep **Section 3 (Common Mistakes)** handy while you practise, and copy **Section 4 (Concept Notes)** by pen for recall & interviews.

1 Learning Plan

Day 9: Production Development — "Work like a professional on a real team"

Today is a little different. It is less about **new syntax** and more about **how real teams build software**. Think of it like your first week joining a professional .NET team: the C# was never the hard part — the hard part was Git, code reviews, folder conventions, and clean service layers. This day gives you those same "team habits" for React + TypeScript.

Your hands-on mission (do it independently): build **one production-quality Employee module end-to-end** — on a feature branch, with a typed API layer, a clean folder structure, small commits, and a Pull Request you open yourself. No hand-holding. This is the dress rehearsal.

> **Heads up: Day 10 is your final assessment.** Today is the last "learning" day before you are tested, so treat this like real work.

Part A — Core Topics

1. Git workflow

Git is your source control — same job as **TFS / Azure DevOps** in .NET. You never build straight on `main`. You create a **feature branch** (a safe copy to work on), make **small commits** (a commit is a saved checkpoint with a message), **push** them to the server, then open a **Pull Request** to merge back. Small commits with clear messages make your history readable — like good check-in comments in TFS.

```
# start from an up-to-date main
git checkout main
git pull

# create a feature branch (branch = your safe working copy)
git checkout -b feature/employee-module

# work, then stage + commit in small steps
git add src/features/employees/api/employeeApi.ts
git commit -m "feat(employees): add typed employee API service"

git add src/features/employees/components/EmployeeList.tsx
git commit -m "feat(employees): render employee list from API"

# push your branch to the server
git push -u origin feature/employee-module
```

A good `.gitignore` keeps junk out of source control (same idea as ignoring `bin/` and `obj/` in .NET):

```
# .gitignore
node_modules/
dist/
.env
.env.*.local
*.log
.DS_Store
```

A **conventional commit** message is a simple naming standard: `type(scope): summary`.

```
# examples of clear, conventional commit messages
feat(employees): add create-employee form
fix(employees): handle 404 when employee id is missing
refactor(api): move error handling into axios interceptor
```

- **.NET / KnockoutJS contrast:** Git feature branch + PR ~ a TFS/Azure DevOps branch + a gated check-in that must pass code review before merge.
- **You'll be able to...** branch, commit in small clear steps, push, and open a PR without touching `main` directly.

---

2. Code reviews

A **Pull Request (PR)** is a request to merge your branch — and it is where a teammate reads your code before it ships. The golden rule: **keep the PR small and well-described.** A reviewer checks a few things: is it **correct**, is it **readable**, are **names** clear, are **edge cases** handled (empty list, error, loading), and are there **tests** where needed. A small PR gets a fast, kind review; a giant one gets ignored or rubber-stamped.

A good PR description tells the reviewer the story so they do not have to guess:

```
Title: feat(employees): add Employee module (list + create)

## What
- New typed API service for employees (list, create)
- EmployeeList and EmployeeForm components

## Why
- First slice of the HR portal Employee feature

## How to test
- Run app, go to /employees, add an employee, see it in the list

## Notes
- Error handling is centralized in the axios interceptor
```

Giving feedback: be kind and specific. Say "*Could we rename* `d` *to* `departments` *for clarity?*" — not "*this is confusing.*" Comment on the **code**, never the person. **Receiving feedback:** it is about the code, not about you. Ask questions, say thanks, and change what makes sense. Every senior dev gets review comments — it is normal.

- **.NET / KnockoutJS contrast:** A PR review ~ your team's code-review gate before a merge — the same checkpoint you already pass in Azure DevOps.
- **You'll be able to...** open a small, clearly-described PR and both give and take feedback like a professional.

---

3. Folder structure

Two common ways to organize files: **type-based** (all components in one folder, all hooks in another) vs **feature-based** (everything for one feature lives together). Feature-based scales better — when you work on "employees," you open **one folder** and find its components, API, hooks, and types side by side. Shared, reusable stuff (buttons, the axios setup) lives in `shared/` and `lib/`.

```
src/
├─ features/
│  └─ employees/                # everything for the Employee feature
│     ├─ components/
│     │  ├─ EmployeeList.tsx
│     │  └─ EmployeeForm.tsx
│     ├─ api/
│     │  └─ employeeApi.ts       # typed calls to the backend
│     ├─ hooks/
│     │  └─ useEmployees.ts      # feature-specific custom hook
│     └─ types/
│        └─ employee.ts          # DTOs / response types
├─ shared/
│  └─ ui/                       # reusable Button, Input, Modal ...
├─ lib/
│  └─ http.ts                   # the shared axios instance
├─ App.tsx
└─ main.tsx
```

- **.NET / KnockoutJS contrast:** Feature folders ~ organizing a .NET solution by **feature/bounded-context** instead of one giant `Controllers/`, `Models/`, `Services/` split — related code stays together.
- **You'll be able to...** lay out a React + TS project so a new teammate finds any feature in seconds.

---

4. API integration

Never call `fetch`/axios scattered inside your components. Build a **typed API service layer** — one clean place that talks to the backend and returns **typed** data. Components and hooks call the service; they never build URLs themselves. This is exactly like a **typed** `HttpClient` **wrapper / repository** in .NET.

First, one shared axios instance with the base URL coming from config:

```
// src/lib/http.ts
import axios from "axios";

// baseURL comes from env config (Vite exposes it via import.meta.env).
// Vite's types for import.meta.env come from `vite/client` (vite-env.d.ts).
export const http = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
  timeout: 10000,
});
```

Then your **DTOs / response types** (a DTO = the shape of data crossing the wire — same word you use in C#):

```
// src/features/employees/types/employee.ts
export interface Employee {
  id: number;
  name: string;
  email: string;
  department: string;
  isActive: boolean;
}

// data we SEND to create an employee — same shape but without the id
export type CreateEmployeeDto = Omit<Employee, "id">;
```

Then the endpoint functions that return **typed** data:

```
// src/features/employees/api/employeeApi.ts
import { http } from "../../../lib/http";
import type { Employee, CreateEmployeeDto } from "../types/employee";

export async function getEmployees(): Promise<Employee[]> {
  const response = await http.get<Employee[]>("/employees");
  return response.data;
}

export async function createEmployee(
  dto: CreateEmployeeDto
): Promise<Employee> {
  const response = await http.post<Employee>("/employees", dto);
  return response.data;
}
```

Now a component just calls `getEmployees()` and gets a fully typed `Employee[]` — no URLs, no `any`, no error plumbing in the UI.

- **.NET / KnockoutJS contrast:** This service layer ~ a typed `HttpClient` wrapper or repository class — the UI depends on typed methods, not raw HTTP.
- **You'll be able to...** build a clean, typed API layer so components stay simple and the backend calls live in one place.

---

5. Coding standards

Professional teams agree on rules so all code looks like it was written by one person. **Prettier** auto-formats (spacing, quotes, semicolons). **ESLint** catches bad patterns (unused vars, `any`, missing keys). Add **strict TypeScript** (`"strict": true`) so the compiler blocks unsafe code. The human rules on top: **clear names, small focused components, no `any`, and DRY** (Don't Repeat Yourself — write shared logic once).

Here is the same component **before** and **after** cleanup:

```
// ❌ before: any types, cryptic names, unclear intent
function C(p: any) {
  return (
    <div>
      {p.d.map((x: any) => (
        <p key={x.id}>{x.n}</p>
      ))}
    </div>
  );
}
```

```
// ✅ after: typed props, clear names, small and focused
interface Department {
  id: number;
  name: string;
}

interface DepartmentListProps {
  departments: Department[];
}

function DepartmentList({ departments }: DepartmentListProps) {
  return (
    <ul>
      {departments.map((dept) => (
        <li key={dept.id}>{dept.name}</li>
      ))}
    </ul>
  );
}
```

- **.NET / KnockoutJS contrast:** ESLint + Prettier ~ **StyleCop / Roslyn analyzers +** `dotnet format` — automated style and quality gates you already know.
- **You'll be able to...** write code that passes lint/format checks and reads clearly to any teammate.

Part B — Extra Topics (deeper professional practice)

B1. API error-handling best practices

Do not scatter `try/catch` in every component, and **never swallow an error silently** (catching it and doing nothing hides bugs). Use an **axios interceptor** — one gate every response passes through — to **normalize** errors (turn messy raw errors into one clean shape) and produce a **user-friendly message**. The UI then just shows the message; it never parses raw HTTP errors.

```
// src/lib/http.ts (add an interceptor to the shared instance)
import axios, { AxiosError } from "axios";

export const http = axios.create({
  baseURL: import.meta.env.VITE_API_URL,
});

http.interceptors.response.use(
  (response) => response,
  (error: AxiosError<{ message?: string }>) => {
    const message =
      error.response?.data?.message ??
      "Something went wrong. Please try again.";
    // convert the raw axios error into a clean Error our UI can show
    return Promise.reject(new Error(message));
  }
);
```

Rules: log the real error for developers, show a friendly message to users, and always re-throw (reject) so the caller knows it failed.

B2. Environment config per environment

Config should change per environment without code changes — same idea as `appsettings.json` / `appsettings.Production.json` in .NET. Vite reads `.env` files. Any variable **must** start with `VITE_` to be visible in the app.

```
# .env  (local development)
VITE_API_URL=http://localhost:5000/api

# .env.production  (used when you build for prod)
VITE_API_URL=https://api.myhrportal.com/api
```

> **CRITICAL — no secrets:** anything with the `VITE_` prefix is **bundled into the browser and is public**. Never put API keys, passwords, or connection strings here. Secrets stay on the **backend**, exactly like you would never ship a C# connection string to a client app.

B3. A quick responsive-UI note

Design **mobile-first** — style for small screens first, then add layout for bigger screens. Use **flexbox/grid** for layout and **relative units** (`rem` , `%` , `fr`) instead of fixed pixels, so the UI stretches nicely.

```
.employee-list {
  display: grid;
  grid-template-columns: 1fr; /* one column on phones */
  gap: 1rem;
}

@media (min-width: 48rem) {
  .employee-list {
    grid-template-columns: 1fr 1fr; /* two columns on wider screens */
  }
}
```

B4. Clean-code mindset + explaining progress to your manager

Clean-code mindset: write for the next human, not the compiler. Small functions, honest names, no dead code, and delete "clever" tricks that need explaining. Code is read far more often than it is written.

Talking to your manager — keep daily updates to a simple 3-part format:

```
Done:     Built the Employee list + create form on a feature branch; PR opened.
Next:     Add edit/delete and error toasts tomorrow.
Blockers: Need the /employees API endpoint enabled on the staging server.
```

Short, factual, and it always states blockers early — managers love predictability, not surprises.

B5. .NET contrast — quick recap

Professional practice (React + TS)	What it is like in .NET
PR review before merge	Your team's code-review gate in Azure DevOps
ESLint + Prettier + strict TS	StyleCop / Roslyn analyzers + `dotnet format`
Typed API service layer	A typed `HttpClient` wrapper / repository
`.env` / `.env.production`	`appsettings.json` / `appsettings.Production.json`

Professional practice (React + TS)	What it is like in .NET
Git feature branch	TFS / Azure DevOps branch

Your Day 9 goal, restated

Build the **Employee module end-to-end, independently**: feature branch → feature folder → typed API service + DTOs → clean components (no `any`) → centralized error handling → small commits → a small, well-described PR. Do this and you are working like a professional React developer — ready for the **Day 10 final assessment**.

2 Detailed Notes — Read Once, Understand Everything

The big idea first

Here is the thing nobody tells you on day one: on a real team, **the code is only half the job.** The other half is *how* you work.

You already know how to make React do things. Day 9 is about doing it the way a professional on a team does it, so that six months from now you (or a teammate) can open the project and instantly understand it.

That "other half" is five habits:

1. **Small, safe Git changes** — so a mistake is easy to undo.
2. **Clear Pull Requests** — so a teammate can review your work before it goes live.
3. **A tidy folder structure** — so everyone can find things.
4. **A clean API layer** — so talking to the server lives in one place, not scattered everywhere.
5. **Consistent standards** — so all our code looks like *one* person wrote it.

You already lived a version of this at your last job. Git is like **TFS / Azure DevOps source control**. A Pull Request is the **code-review gate** before code ships. Coding standards are like **StyleCop / Roslyn analyzers** nagging you into a house style. The API layer is like a **typed `HttpClient` wrapper / repository**. Same ideas, new tools. Let's go through them.

Today's hands-on framing: you will build one production-quality **Employee module**, end to end, on your own. (Day 10 is your final assessment — so treat today as the dress rehearsal.)

---

1) Git workflow — small, safe steps

Git is source control (it saves the history of your code). A **repository** ("repo") is just the project folder Git is watching. The key idea is: never work directly on `main`. Instead, make a **branch**.

A **branch** is your own private copy of the code to work on. It *isolates* your work (keeps it separate) so you can experiment freely without breaking what everyone else sees. In TFS terms, think of it as your own safe workspace that no one else touches until you're ready.

A **commit** is one saved snapshot of your changes. Make them **small** — one small idea per commit. Small commits are easy to review and easy to *revert* (undo) if something goes wrong. A giant commit with 40 changed files is a nightmare to review; ten tiny commits tell a story.

A good **commit message** says **what** you did and **why**. The "why" is the gold — the code already shows the "what".

```
# 1. Start from an up-to-date main (like "Get Latest" in Azure DevOps)
git checkout main
git pull

# 2. Make a branch for YOUR task only — this isolates your work
git checkout -b feature/employee-list

# 3. Do a little work, then commit small (easy to review, easy to revert)
git add src/features/employees/
git commit -m "feat(employees): add typed employee list with loading state

Why: the list was fetched inline with 'any', so typos slipped through.
Moved it to a typed service + hook so the UI stays simple and type-safe."

# 4. Before you push, pull the latest main so you don't fall behind
git pull origin main --rebase

# 5. Push your branch up to the server
git push -u origin feature/employee-list

# 6. Open a Pull Request on GitHub / Azure DevOps → review → merge to main
```

Always pull before you push. Someone else may have merged work while you were coding. Pulling first means you deal with any overlap on *your* machine, calmly, instead of pushing a mess.

The **Pull Request (PR)** is the review gate. It says: "here are my changes — please review before we merge them into `main`." Nothing reaches `main` without passing through a PR. This is exactly the code-review gate you know from Azure DevOps.

Finally, a sane `.gitignore` — a list of files Git should *never* save. You don't commit build junk or secrets:

```
# .gitignore — files Git must NEVER track
node_modules/     # installed packages: huge, and easily restored with `npm install`
dist/             # build output: generated, so no need to save it
.env              # secrets and config — NEVER commit these
*.local
.DS_Store
```

---

2) Code reviews — the quality gate

A code review is a teammate reading your PR before it merges. Your job is to make their job **easy**.

How to open a PR people enjoy reviewing:

- **Small.** A PR that touches 3 files gets a careful review. One that touches 30 gets a lazy "looks good" (which helps no one).
- **Focused.** One PR = one thing. Don't fix a bug *and* rename ten files *and* add a feature in the same PR.
- **Described.** Write a short description: what changed, why, and how to test it. Add a screenshot for UI changes.
- **Self-reviewed first.** Read your own diff before asking anyone else. You'll catch half the issues yourself — a leftover `console.log`, a bad name, a commented-out block.

What the reviewer is looking for (keep this checklist in your head — it's your StyleCop-for-humans):

- Does it actually **work** and solve the task?
- Is it **readable**? Could a new teammate follow it?
- Are the **names** clear (`activeEmployees`, not `data2`)?
- Are the **edge cases** handled — empty list, loading, error, `null` ?
- Are there **tests** where they matter?
- Any **leftovers** — `console.log`, dead code, `// TODO fix later` ?
- Any **secrets** — API keys, passwords, tokens? (These must never be in code.)

Giving feedback — be specific and kind. Point at the exact line and suggest, don't command. "What do you think about pulling this into a helper so we don't repeat it?" lands far better than "this is wrong." Praise the good parts too.

Receiving feedback — it's help, not an attack. The reviewer is improving the *code*, not judging *you*. Say thanks. If you don't understand a comment, ask "why?" — it's a chance to learn. Every senior dev you admire has had thousands of comments on their PRs. It's normal.

---

3) Folder structure — where things live

Two ways to organize a React project:

- **Type-based** — group by *kind*: all components in `components/`, all hooks in `hooks/`, all services in `services/`. Fine for a tiny app. But as it grows, one feature is smeared across five far-apart folders. To change "employees" you jump all over the tree.
- **Feature-based** — group by *feature*: everything for employees lives in `features/employees/`. Its components, hooks, API calls, and types all sit together.

Think of it like organizing a company. Type-based is "put all managers in one building and all developers in another." Feature-based is "each product team sits together in one room." The second one scales — it's the same instinct as **vertical-slice / feature folders** in .NET, versus stacking everything by layer.

Feature-based wins because everything you need for a change is **in one place**, features don't tangle together, and you can delete a whole feature by deleting one folder.

```
src/
├─ app/                 # app shell: routing, providers, top-level layout
│  └─ App.tsx
├─ shared/              # reusable, feature-agnostic building blocks
│  ├─ ui/               # dumb UI: Button, Input, Modal
│  ├─ hooks/            # generic hooks: useDebounce, useToggle
│  └─ lib/              # apiClient and small helpers
├─ config/
│  └─ env.ts            # typed configuration read from environment variables
└─ features/            # one folder per feature (the important part)
   └─ employees/
      ├─ api/           # employeesApi.ts   → talks to the server
      ├─ hooks/         # useEmployees.ts   → orchestrates state
      ├─ components/    # EmployeeList.tsx, EmployeeCard.tsx → render UI
      ├─ types.ts       # EmployeeDto, CreateEmployeeDto
      └─ index.ts       # a "barrel" that re-exports the public bits
```

Keep truly shared, reusable pieces in `shared/ui` and `shared/lib` — things any feature can use. If it only makes sense for employees, it stays inside `features/employees/`.

That `index.ts` is a **barrel export** — a small file that re-exports a feature's public pieces, like `export { EmployeeList } from "./components/EmployeeList";`. It's a tidy front door (a bit like a namespace facade in C#). Now other code writes a short `import { EmployeeList } from "@/features/employees"` instead of a long, fragile deep path.

---

4) API integration — a clean, typed service layer

This is the heart of professional React, so slow down here. The rule is **separation of concerns** — every layer has one job:

- **Components render.** They show data and fire events. They should be "dumb and pretty."
- **Hooks orchestrate.** They track loading/error/data and decide *when* to fetch.
- **The API / service layer talks to the server.** All the HTTP lives here.

Scattering `fetch` calls directly inside components is the classic beginner trap. When the base URL changes, or you need to add an auth header, or handle errors consistently, you're editing twenty files. Instead, build it in layers — exactly like a typed `HttpClient` wrapper or repository in .NET.

First, config from the environment. An **environment variable** is a setting that lives *outside* your code (in a `.env` file), so the same code can point at localhost while developing and at the real server in production. Vite exposes them on `import.meta.env`. This tiny declaration file just tells TypeScript they exist, so they're typed as `string`, not `any`:

```
// src/vite-env.d.ts
// Vite creates this file. It just TELLS TypeScript which env vars exist,
// so import.meta.env.VITE_API_BASE_URL is typed as string (not any).
interface ImportMetaEnv {
  readonly VITE_API_BASE_URL: string;
}

interface ImportMeta {
  readonly env: ImportMetaEnv;
}
```

```
// src/config/env.ts
// Read configuration in ONE typed place — like IConfiguration in ASP.NET.
// If a value is missing or wrong, you fix it HERE, not in twenty components.
export const env = {
  apiBaseUrl: import.meta.env.VITE_API_BASE_URL,
} as const;
```

Next, one shared HTTP client. We create a single **axios** instance with the base URL baked in, plus one **interceptor** — a hook that runs on *every* response, so we handle errors in one spot (like a global exception filter / middleware in ASP.NET):

```ts
// src/shared/lib/apiClient.ts
import axios, { AxiosError, type AxiosInstance } from "axios";
import { env } from "../../config/env";

// One shared, pre-configured client — like a single typed HttpClient
// registered in your DI container and reused everywhere.
export const apiClient: AxiosInstance = axios.create({
  baseURL: env.apiBaseUrl,
  timeout: 10_000,
  headers: { "Content-Type": "application/json" },
});

// One interceptor = one place to handle EVERY failed call.
apiClient.interceptors.response.use(
  (response) => response,
  (error: AxiosError<{ message?: string }>) => {
    const message =
      error.response?.data?.message ?? error.message ?? "Something went wrong.";
    console.error(`[API] ${error.response?.status ?? "network"} — ${message}`);
    return Promise.reject(new Error(message));
  },
);
```

Then, describe the data with types. A **DTO** (Data Transfer Object) is the exact shape the server sends or receives — the same idea as a C# `record` /POCO you deserialize JSON into:

```ts
// src/features/employees/types.ts

// DTO = Data Transfer Object: the exact shape the server sends/receives.
export interface EmployeeDto {
  id: number;
  firstName: string;
  lastName: string;
  email: string;
  department: string;
  isActive: boolean;
}

// What we SEND when creating a new employee (no id yet — the server assigns it).
export type CreateEmployeeDto = Omit<EmployeeDto, "id">;

// A reusable envelope, if your API wraps lists with paging info.
export interface PagedResult<T> {
  items: T[];
  total: number;
}
```

Now the endpoint functions. Each one talks to *one* endpoint and returns a *typed* result. This file is your typed repository:

```
// src/features/employees/api/employeesApi.ts
import { apiClient } from "../../../shared/lib/apiClient";
import type { CreateEmployeeDto, EmployeeDto } from "../types";

export async function getEmployees(): Promise<EmployeeDto[]> {
  const response = await apiClient.get<EmployeeDto[]>("/employees");
  return response.data;
}

export async function getEmployeeById(id: number): Promise<EmployeeDto> {
  const response = await apiClient.get<EmployeeDto>(`/employees/${id}`);
  return response.data;
}

export async function createEmployee(
  payload: CreateEmployeeDto,
): Promise<EmployeeDto> {
  const response = await apiClient.post<EmployeeDto>("/employees", payload);
  return response.data;
}
```

Finally, the hook that orchestrates. It calls the service, tracks loading and errors, and hands clean, typed data to the component — which stays simple:

```ts
// src/features/employees/hooks/useEmployees.ts
import { useCallback, useEffect, useState } from "react";
import { getEmployees } from "../api/employeesApi";
import type { EmployeeDto } from "../types";

interface UseEmployeesResult {
  employees: EmployeeDto[];
  isLoading: boolean;
  error: string | null;
  reload: () => void;
}

export function useEmployees(): UseEmployeesResult {
  const [employees, setEmployees] = useState<EmployeeDto[]>([]);
  const [isLoading, setIsLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);

  const load = useCallback(async () => {
    setIsLoading(true);
    setError(null);
    try {
      const data = await getEmployees();
      setEmployees(data);
    } catch (err) {
      // catch is 'unknown' — narrow it before reading .message
      setError(err instanceof Error ? err.message : "Failed to load employees.");
    } finally {
      setIsLoading(false);
    }
  }, []);

  useEffect(() => {
    void load();
  }, [load]);

  return { employees, isLoading, error, reload: () => void load() };
}
```

See the payoff? The base URL, headers, and error handling live in **one** place. Change the server address once. Add an auth token once. The component just calls `useEmployees()` and renders. That is why this beats `fetch` scattered across components.

5) Coding standards — so anyone can read your code

The goal: make all our code look like **one careful person** wrote it. Two tools do the boring work for you:

- **ESLint** — a *linter* (a tool that reads your code and warns about likely bugs and bad patterns). This is your **Roslyn analyzer**. Example: it flags `any`, unused variables, and missing `useEffect` dependencies.
- **Prettier** — a *formatter* that auto-fixes spacing, quotes, and line breaks on save. No more arguing about style in PRs — the machine decides.

Add a few human rules on top:

- **Strict TypeScript, and no `any`.** `any` switches off type-checking — it throws away the safety you came to TypeScript for. Give things real types.
- **Clear names.** `activeEmployees`, not `arr`. `isLoading`, not `flag`.
- **Small, focused components.** If a component fetches, filters, formats, *and* renders, split it up. One job each.
- **DRY** ("Don't Repeat Yourself") — if you copy-paste logic, pull it into a shared function or hook.
- **Consistent error handling** — one pattern everywhere (we centralized it in the interceptor above).

A short before/after makes it concrete:

```tsx
// src/features/employees/components/EmployeeList.tsx
import { useEffect, useState, type ReactElement } from "react";

interface Employee {
  id: number;
  firstName: string;
  lastName: string;
}

// BEFORE: one big component that fetches, uses 'any', and ignores loading/errors.
export function EmployeeListBad(): ReactElement {
  // 'any[]' turns off type-checking, so a typo like r.frstName won't be caught.
  const [rows, setRows] = useState<any[]>([]);
  useEffect(() => {
    void fetch("http://localhost:5000/api/employees")
      .then((res) => res.json())
      .then((data: any[]) => setRows(data));
  }, []);
  return (
    <ul>
      {rows.map((r) => (
        <li key={r.id}>{r.firstName}</li>
      ))}
    </ul>
  );
}

// AFTER: a small, typed, "dumb" component. It only renders what it is given.
// Fetching + loading + errors now live in the hook + service (Section 4).
export function EmployeeList({
  employees,
}: {
  employees: Employee[];
}): ReactElement {
  return (
    <ul>
      {employees.map((employee) => (
        <li key={employee.id}>
          {employee.firstName} {employee.lastName}
        </li>
      ))}
```

```
    </ul>
  );
}
```

The "after" is shorter, fully typed, and does *one* thing. That's the whole standard in a nutshell.

---

6) A few more pro habits

Environment config. Keep settings in a `.env` file, never hard-coded and never committed (remember the `.gitignore`). In Vite, only variables that start with `VITE_` are exposed to the browser — a safety rail so you don't accidentally leak a secret. Same spirit as `appsettings.json` plus `appsettings.Production.json` in .NET.

Responsive UI basics. "Responsive" just means the layout works on a phone *and* a big monitor. A few easy wins: use **flexbox / CSS grid** to arrange things, use relative units (`rem`, `%`) instead of fixed pixels for widths, add `max-width: 100%` on images, and test by dragging your browser narrow. Build mobile-first — start small, then add space as the screen grows.

Reporting progress to your manager. Be brief and honest. A great daily update has three lines: **Done** (what I finished), **Doing** (what I'm on now), **Blockers** (anything stopping me). For example: "*Done: employee list PR merged. Doing: the create-employee form. Blocker: waiting on the final API field names.*" Managers love clarity, and they especially love hearing about blockers *early* — not the day the deadline slips.

One-line summary

Working like a pro means the code is only half of it — the other half is small safe Git changes, clear Pull Requests, a feature-based folder structure, a clean typed API layer, and consistent standards, so anyone on the team can read and trust your work.

3 Common Mistakes to Avoid

On Day 9 the mistakes are less about React syntax and more about *how you work on a team*. Here are the common ones, in plain ❌ / ✅ form. (Day 10 is your final assessment, so getting these habits right now really pays off.)

3.1 Git basics

❌ **Committing straight to `main`.** `main` is the shared, "always working" branch. Pushing half-finished work there breaks it for everyone — like editing shared source in TFS/Azure DevOps with no shelveset and no branch.

✅ **Make a feature branch for every task.** Think of it like a task branch in Azure DevOps.

```
# ❌ working straight on main
git checkout main
git commit -m "stuff"

# ✅ one branch per feature, then push it up
git checkout -b feature/employee-create-form
# ... small commits ...
git push -u origin feature/employee-create-form
```

❌ **Giant "big bang" commits** (50 files, three features, one commit). Nobody can review or undo them.
✅ **Small, focused commits** — one logical change each. Easy to read, easy to roll back.

❌ **Vague messages** like `fix`, `wip`, `changes`, `asdf`. Six months later nobody knows what happened.
✅ **Clear messages** that say *what* changed — like a good check-in comment.

```
# ❌ tells the team nothing
git commit -m "fix"
git commit -m "wip"

# ✅ says what changed, in plain words
git commit -m "Add validation to employee create form"
```

❌ **Committing `node_modules`, `.env`, or secrets (API keys, passwords, connection strings).** `node_modules` is huge and rebuildable. `.env`/secrets in Git means anyone with repo access has your keys — and Git remembers them forever, even after you delete them. ✅ **Add a `.gitignore`, and keep real secrets on the server / in a secrets store** (like using `appsettings` + environment variables in .NET instead of hardcoding a connection string).

```
# .gitignore — keep these OUT of source control
node_modules/
.env
.env.local
*.local
dist/
```

3.2 Pull Requests & code reviews

A Pull Request (PR) is the code-review gate before your branch merges into `main` — same idea as a required reviewer on a pull request in Azure DevOps.

❌ **Opening a huge PR with 40 files and no description.** Reviewers get lost and either rubber-stamp it or leave it sitting for days. ✅ **Keep PRs small, and write a short description**: what it does, why, and how to test it. Small PR = fast, real review.

❌ **Not self-reviewing first.** You send it straight to the team with a stray `console.log`, a leftover comment, or a typo still in it. ✅ **Read your own "Files changed" tab before you ask anyone else.** You'll catch half the issues yourself.

❌ **Taking review comments personally.** A comment like "extract this into a helper" is not an attack — and getting defensive slows the whole team down. ✅ **Remember: the feedback is about the code, not about you.** Reviews are how the team keeps quality high. Ask questions if a comment is unclear, thank people, and move on. (And when *you* review, be kind — critique the code, not the person.)

3.3 Folder structure

❌ **Everything dumped in one folder** (`src/` with 60 files). Nobody can find anything.

❌ **Grouping only by type**, so one feature is scattered across four folders. To change "Employee" you jump between `components/`, `hooks/`, `services/`, and `types/`:

```
src/
  components/   EmployeeList.tsx  EmployeeCard.tsx  LoginForm.tsx  ...
  hooks/        useEmployee.ts    useAuth.ts         ...
  services/     employeeApi.ts    authApi.ts         ...
  types/        employee.ts       auth.ts            ...
```

✅ **Group by feature.** Everything for "Employee" lives together, and truly shared things go in `shared/`. Deleting or moving a feature is easy because it's all in one place:

```
src/
  features/
    employees/
      components/   EmployeeList.tsx  EmployeeCard.tsx
      hooks/        useEmployee.ts
      api/          employeeApi.ts
      types.ts
  shared/           Button.tsx  useToast.ts     (used by many features)
  app/              routes, providers, layout
```

❌ **Too-deep nesting** (`src/features/employees/components/list/rows/cells/ ...`). If you dread typing the import path, it's too deep. ✅ **Keep it flat enough to navigate** — a couple of levels inside a feature is plenty.

3.4 API integration

❌ `fetch` / `axios` **calls scattered directly inside components**, with hardcoded URLs, responses typed as `any`, and errors quietly ignored. That's like `new HttpClient()` and building URLs by hand in every button click of your C# app.

✅ **Put all API calls in one typed service layer**, separate from the UI. One shared client, base URL from an env variable, typed responses, and real error handling:

```
import axios from "axios";

// A clean, typed API service - kept OUT of the components.
export interface Employee {
  id: number;
  name: string;
  department: string;
}

// One shared axios instance. In a real app the base URL comes from an
// env variable, e.g. baseURL: import.meta.env.VITE_API_URL
const api = axios.create({ baseURL: "/api" });

function showError(message: string): void {
  // in a real app this shows a toast/banner to the user
  console.error(message);
}

// Typed return value, and errors are handled (never swallowed).
export async function getEmployee(id: number): Promise<Employee | null> {
  try {
    const res = await api.get<Employee>(`/employees/${id}`);
    return res.data;
  } catch (err) {
    console.error(err);
    showError("Could not load employee. Please try again.");
    return null;
  }
}
```

❌ **Hardcoded URLs** like `https://api.myapp.com/...` copied into every call. When the URL changes (or you move to staging), you must hunt down every one. ✅ **Set the base URL once, from an env variable** (`VITE_API_URL`) — one place to change, like `appsettings.json` per environment in .NET.

❌ **Typing responses as `any`.** You throw away everything TypeScript gives you — no autocomplete, no safety, crashes at runtime. ✅ **Describe the response shape with a type.** Wrong fields and typos get caught while you code.

```
// ❌ typing data as `any` switches TypeScript's safety OFF
const loose: any = JSON.parse("{}");
console.log(loose.user.name.first); // compiles fine, but can crash at runtime

// ✅ describe the shape with a type - mistakes get caught early
interface LoginResponse {
  token: string;
  expiresIn: number;
}
const safe = JSON.parse("{}") as LoginResponse;
console.log(safe.token);
```

❌ **Swallowing errors** with an empty `catch {}`. The call fails, and the user just stares at a spinner forever. ✅ **Always handle failure**: log it, and show the user a clear message (a toast, a banner, an inline error) — like showing a friendly error instead of a raw 500 in an MVC app.

3.5 Coding standards

ESLint + Prettier are your automatic style/quality checkers — the JS/TS version of StyleCop and Roslyn analyzers.

❌ **Ignoring ESLint/TypeScript errors**, or turning off `strict` mode to "make the red squiggles go away." You're just hiding real bugs. ✅ **Fix the warnings, keep `strict: true` on.** Strict mode is what makes TypeScript feel like C#'s type safety.

❌ **Overusing `any`** to silence the compiler. One `any` quietly spreads and disables checking everywhere it flows. ✅ **Use a real type, or `unknown` + a check** when you genuinely don't know the shape yet.

❌ **Inconsistent naming** — `EmpList`, `employee_card`, `getdata`, all mixed together. ✅ **Pick one convention and stick to it**: `PascalCase` for components and types, `camelCase` for variables and functions. Consistent naming reads like consistent C# conventions across a solution.

❌ **Copy-pasting the same code** into five components. Fix a bug once, and it stays broken in the other four. ✅ **Extract a shared component or custom hook** and reuse it (DRY — Don't Repeat Yourself). Same instinct as pulling repeated C# logic into a shared method or base class.

3.6 One golden rule: never rewrite shared history

❌ **Force-pushing a shared branch** (`git push --force origin main`). This rewrites history for everyone and can wipe out teammates' work — one of the fastest ways to make the whole team unhappy.

```
# ❌ NEVER on a shared branch — it overwrites everyone else's history
git push --force origin main

# ✅ if you must overwrite YOUR OWN feature branch, use the safer flag
#    (it refuses to push if someone else pushed there first)
git push --force-with-lease origin feature/my-branch
```

✅ **Rule of thumb:** never force-push or rebase a branch other people share. Do that kind of tidy-up only on your own branch, before anyone else has pulled it.

4 Concept Notes — Recall & Interview (copy by pen)

> These are your quick-recall notes for Day 9 — read them to lock in the ideas and to warm up before interviews.

Git & Version Control

Version Control / Git

Definition:

- A tool that tracks every change to your code, so you can go back in time and work safely as a team.
- Like Azure DevOps / TFS source control in .NET — but Git keeps a full copy of the history on your own machine.

✅ **Advantages:**

- Full history; safe to experiment; easy teamwork.

❌ **Disadvantages / watch-outs:**

- Small learning curve; history gets messy if commits are careless.

📝 **Example:**

- `git status` — shows what files you changed.

Feature-Branch Workflow

Definition:

- You make a new branch for each feature or fix, keep `main` clean, then merge back when it is done.
- A branch is just a separate line of work — a private copy you can change without breaking others.

✅ **Advantages:**

- `main` stays stable; many people work at once; easy to review one feature.

❌ **Disadvantages / watch-outs:**

- Long-lived branches drift far from `main` and cause big merge conflicts — keep branches short.

💡 **Interview tip:**

- Feature branch = isolate each change; trunk-based = commit tiny changes straight to `main` behind flags. Feature branches are easier for small teams and reviews.

📝 **Example:**

- `git checkout -b feature/employee-list` — start a new feature branch.

Commit

Definition:

- A commit is one saved snapshot of your changes with a short message saying what changed and why.
- Keep commits small and focused — one idea per commit.

✅ **Advantages:**

- Easy to read the history; easy to undo one small change.

❌ **Disadvantages / watch-outs:**

- Huge "fixed stuff" commits are hard to review or revert.

📝 **Example:**

- `git commit -m "Add employee list table"` — save a snapshot with a clear message.

Push / Pull

Definition:

- Push sends your local commits up to the shared server; pull brings other people's commits down to you.
- Pull often so you don't drift from the team.

✅ **Advantages:**

- Keeps everyone in sync; your work is backed up on the server.

❌ **Disadvantages / watch-outs:**

- Forgetting to pull leads to conflicts; pushing broken code blocks others.

📝 **Example:**

- `git pull` then `git push` — get the latest, then send yours.

Pull Request (PR)

Definition:

- A Pull Request asks the team to review and merge your branch into `main`.
- Like a code-review gate in Azure DevOps — nothing merges until someone approves.

✅ **Advantages:**

- Catches bugs early; shares knowledge; keeps quality high.

❌ **Disadvantages / watch-outs:**

- Giant PRs are slow to review — keep them small and focused.

💡 **Interview tip:**

- A good PR is small, has a clear title and description, links the ticket, explains the "why", and passes CI (the automated checks) before review.

📝 **Example:**

- `gh pr create --fill` — open a PR from the current branch.

Merge vs Rebase

Definition:

- Merge joins two branches and keeps both histories (adds a merge commit); rebase replays your commits on top of the latest `main` to give a straight line.
- Safe advice: merge for shared branches; rebase only your own local, un-pushed work.

✅ **Advantages:**

- Merge is safe and honest about history; rebase gives a clean, linear history.

❌ **Disadvantages / watch-outs:**

- Never rebase commits others have already pulled — it rewrites history and breaks their copy.

💡 **Interview tip:**

- Golden rule: "rebase private branches, merge public ones." Don't rebase shared history.

📝 **Example:**

- `git rebase main` — replay your branch on top of the latest `main`.

Code Review

Definition:

- Teammates read your PR and give feedback before it merges.
- Reviewers look for correctness, readability, tests, naming, and edge cases — not just style.

✅ **Advantages:**

- Fewer bugs; shared standards; everyone learns.

❌ **Disadvantages / watch-outs:**

- Keep feedback kind and about the code, not the person; don't nitpick endlessly.

💡 **Interview tip:**

- Giving feedback: be specific and kind, suggest don't demand. Receiving it: don't take it personally, and reply to every comment.

📝 **Example:**

- `"Nit: rename data to employees for clarity"` — a small, kind suggestion.

Project Structure & Architecture

Folder Structure — Feature-based vs Type-based

Definition:

- Type-based groups files by kind (all components together, all hooks together); feature-based puts everything for one feature in one folder.
- Feature-based scales better as the app grows.

✅ **Advantages:**

- Feature-based: everything for a feature lives together; easy to find and easy to delete.

❌ **Disadvantages / watch-outs:**

- Type-based spreads one feature across many folders; painful in big apps.

💡 **Interview tip:**

- Prefer feature-based for large apps: high cohesion (related files together), low coupling (features don't lean on each other).

📝 **Example:**

- `features/employees/{components,hooks,api,types}` — one feature, all its parts.

Separation of Concerns

Definition:

- Each part of the code does one job — UI shows data, the API layer fetches it, hooks hold the logic.
- Keeps things easy to change and test.

✅ **Advantages:**

- Change one layer without breaking the others; easier testing.

❌ **Disadvantages / watch-outs:**

- Over-splitting a tiny app adds needless files — match the structure to the size.

📝 **Example:**

- A component calls `getEmployees()` — the UI never knows about axios.

API / Service Layer

Definition:

- One place that holds all your HTTP calls, returns typed data, and stays out of the components.
- Like a typed HttpClient wrapper or repository in .NET.

✅ **Advantages:**

- Reuse calls; change the URL or headers in one spot; components stay clean.

❌ **Disadvantages / watch-outs:**

- Don't let axios/fetch leak into components — always go through the service.

💡 **Interview tip:**

- Separate the API layer so the UI doesn't depend on how data is fetched; you can swap axios, add caching, or mock it in tests easily.

📝 **Example:**

- `getEmployees = () => api.get<Employee[]>('/employees')` — one typed endpoint function.

DTO / Typed Responses

Definition:

- A DTO (Data Transfer Object) is a type that describes the exact shape of the data coming from the API.
- Same idea as a C# DTO / response model.

✅ **Advantages:**

- Autocomplete and compile-time safety; you catch a wrong field name before runtime.

❌ **Disadvantages / watch-outs:**

- Keep DTOs in sync with the backend; don't blindly reuse them as UI state.

📝 **Example:**

- `interface Employee { id: number; name: string }` — the response shape.

Error Handling (Interceptors)

Definition:

- An interceptor is code that runs on every request or response — the perfect spot for one central error handler.
- Handle errors once, not in every component.

✅ **Advantages:**

- One place to log the error, show a toast, or redirect on a 401 (not-logged-in) response.

❌ **Disadvantages / watch-outs:**

- Don't swallow errors silently; still let the caller know when it matters.

📝 **Example:**

- `api.interceptors.response.use(ok, handleError)` — one central catch for all calls.

Environment Variables

Definition:

- Config values (like the API base URL) that live outside your code and change per environment (dev, test, prod).
- Like appsettings.json / environment config in .NET.

✅ **Advantages:**

- One build works everywhere; no hard-coded URLs.

❌ **Disadvantages / watch-outs:**

- Front-end env vars are PUBLIC — they ship to the browser, so never put secrets or passwords in them.

📝 **Example:**

- `import.meta.env.VITE_API_URL` — read the base URL from env (Vite).

Coding Standards & Quality

Coding Standards (ESLint + Prettier)

Definition:

- ESLint catches bad or risky code; Prettier auto-formats it so everyone's code looks the same.
- Like Roslyn analyzers / StyleCop plus `dotnet format` in .NET.

✅ **Advantages:**

- Consistent code; fewer silly bugs; no arguments about spacing or style.

❌ **Disadvantages / watch-outs:**

- Don't disable a rule just to "make it pass" — fix the real issue.

📝 **Example:**

- `npm run lint` — check the code against the rules.

Strict TypeScript / Avoiding `any`

Definition:

- Strict mode turns on TypeScript's full safety checks; `any` turns them off, so avoid it.
- Use a real type, or `unknown` when you're not sure yet.

✅ **Advantages:**

- Catches bugs at compile time; great autocomplete.

❌ **Disadvantages / watch-outs:**

- `any` spreads and silently disables checks — it's the opposite of why you use TypeScript.

💡 **Interview tip:**

- Avoid `any` because it removes all type safety; prefer `unknown` (which forces you to check) or a proper type.

📝 **Example:**

- `"strict": true` in tsconfig — turn on all the safety checks.

DRY (Don't Repeat Yourself)

Definition:

- Write logic once and reuse it, instead of copy-pasting the same code around.

✅ **Advantages:**

- Fix a bug in one place; less code to maintain.

❌ **Disadvantages / watch-outs:**

- Don't force-share things that just look similar — a wrong abstraction hurts more than a little duplication.

📝 **Example:**

- One `<Button>` used everywhere — not ten copies.

Clean, Small Components

Definition:

- Each component does one thing and stays short; push logic into hooks and helper functions.

✅ **Advantages:**

- Easy to read, test, and reuse.

❌ **Disadvantages / watch-outs:**

- Giant "do-everything" components are hard to change and full of bugs.

📝 **Example:**

- `<EmployeeList>` renders the rows; a `useEmployees()` hook does the fetching.